МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность ПО-12

Выполнил

Марчук М. Т,

студент групп ПО-12

Проверила

Кулик А.Д,

Брест 2026

Цель работы: Приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1 (Порождающий паттерн: Фабричный метод)

Реализовать проект «Кофе-автомат».

```python
"""

Модуль для реализации паттерна 'Фабричный метод'.

Обеспечивает создание различных видов кофе через единый интерфейс.

"""

from abc import ABC, abstractmethod




class Coffee(ABC):

    """Абстрактный класс продукта кофе."""


    @abstractmethod

    def get_name(self) -> str:

        """Возвращает название напитка."""




class Espresso(Coffee):

    """Класс напитка Эспрессо."""


    def get_name(self) -> str:
```

```
        """Реализация получения названия."""
        return "Эспрессо"



class Latte(Coffee):
    """Класс напитка Латте."""

    def get_name(self) -> str:
        """Реализация получения названия."""
        return "Латте"



class CoffeeMachine:
    """Класс-фабрика для управления заказами."""

    @staticmethod
    def order_coffee(coffee_type: str) -> Coffee:
        """
        Создает объект кофе на основе выбора пользователя.
        :param coffee_type: Строка с кодом напитка
        :return: Объект класса Coffee
        """
        menu = {
```

```
            "1": Espresso(),
            "2": Latte()
        }
        return menu.get(coffee_type, Espresso())


def main():
    """Точка входа в программу."""
    print("--- Кофе-автомат ---")
    user_choice = input("Выберите кофе (1-Эспрессо, 2-Латте): ")
    drink = CoffeeMachine.order_coffee(user_choice)
    print(f"Ваш напиток: {drink.get_name()}")


if __name__ == "__main__":
    main()
```

Задание 2 (Структурный паттерн: Адаптер)

Реализовать проект «Часы» (аналоговые и цифровые).

```
"""
Модуль для реализации паттерна 'Адаптер'.
Позволяет использовать аналоговые часы как цифровые.
```

```python
"""




class AnalogClock:
    """Класс оригинальных аналоговых часов."""

    def __init__(self, hours_angle: float):
        """Инициализация угла стрелки."""
        self.angle = hours_angle




class DigitalClockAdapter:
    """Класс-адаптер для преобразования формата времени."""

    def __init__(self, analog_clock: AnalogClock):
        """Принимает объект аналоговых часов."""
        self.analog_clock = analog_clock

    def get_digital_time(self) -> str:
        """
        Преобразует градусы в часы.
        :return: Строка времени HH:MM
        """
```

```
        # 30 градусов = 1 час
        hours = int(self.analog_clock.angle // 30)
        return f"{hours:02d}:00"



def main():
    """Точка входа в программу."""
    print("--- Адаптер часов ---")
    # 90 градусов соответствуют 3 часам
    analog = AnalogClock(90.0)
    adapter = DigitalClockAdapter(analog)
    print(f"Цифровое время на дисплее: {adapter.get_digital_time()}")



if __name__ == "__main__":
    main()
```

Задание 3 (Поведенческий паттерн: Стратегия)

Реализовать проект «Клавиатура настраиваемого калькулятора».

```
"""
Модуль для реализации паттерна 'Стратегия'.
Позволяет изменять функционал кнопок калькулятора во время выполнения.
```

```python
"""

import math

from abc import ABC, abstractmethod




class ButtonFunction(ABC):

    """Абстрактный класс стратегии функции кнопки."""


    @abstractmethod

    def calculate(self, value: float) -> float:

        """

        Выполняет расчет.

        :param value: Входное число.

        """




class SqrtStrategy(ButtonFunction):

    """Стратегия для вычисления квадратного корня."""


    def calculate(self, value: float) -> float:

        """Расчет корня."""

        return math.sqrt(value)
```

```python
class CalculatorButton:
    """Класс кнопки калькулятора, использующий стратегию."""

    def __init__(self, strategy: ButtonFunction):
        """
        Инициализация с конкретной стратегией.
        :param strategy: Объект функции кнопки.
        """
        self.strategy = strategy

    def press(self, value: float) -> float:
        """
        Нажатие на кнопку.
        :param value: Число для обработки.
        :return: Результат вычисления.
        """
        return self.strategy.calculate(value)


def main():
    """Точка входа в программу."""
    button = CalculatorButton(SqrtStrategy())
```

```
    print(f"Корень из 16: {button.press(16.0)}")




if __name__ == "__main__":

    main()
```

Выводы: Использование паттернов проектирования позволило сделать архитектуру программ более гибкой. Паттерны помогают следовать принципам SOLID, делая систему расширяемой без изменения существующего кода.